VHDL – Lab 6

Testbench and Debug

Address:
Mor Kerstins väg 27
511 53 Kinna
Sweden

Tel:
+46-703-748429

Registration no:
559037-6603

E-mail:
kent@elekteknik.se

Revision history

Date	Author	Description
2023-04-25	Kent Abrahamsson	First edition

Address:
Mor Kerstins väg 27
511 53 Kinna
Sweden

Tel:
+46-703-748429

Registration no:
559037-6603

E-mail:
kent@elekteknik.se

1 Table of Contents

1 TABLE OF CONTENTS 3
2 GENERAL 4
2.1 Background 4
2.2 Intended outcome 4
2.3 Checkpoints 4
3 LAB INSTRUCTIONS 5
3.1 General 5
3.2 Running a testbench in Modelsim 8

Address:
Mor Kerstins väg 27
511 53 Kinna
Sweden

Tel:
+46-703-748429

Registration no:
559037-6603

E-mail:
kent@elekteknik.se

2 General

2.1 Background

When designs grow in complexity we want to have an automated way of testing them. We do not want to force signals in the transcript window (or using .do-files) and manually check the waveform window to ensure that everything performs as required. For larger designs this approach will end up in chaos!

To test designs automatically we can use testbenches. The testbench is a behavioral VHDL file with the design under test (DUT) instantiated as a sub component. One or often several other test processes or test components are used to create stimuli and check outputs from the DUT.

In this lab you will learn how to handle a testbench using a main test process without sensitivity list, built using wait statements. This type of VHDL is a little different from what you're used to since the "normal" synchronous process is typically triggered every clock cycle. You will be given a testbench base file to get started.

The simulation is, as the name implies, a simulation of the reality and sometimes there are differences between the simulation and the real hardware that needs to be measured. One part of this lab is to edit the design and introduce debug signals in order to measure how long the start bit of the serial communication is. This debug strategy is often called "code instrumentation", i.e. adding code to measure things that may be hard to measure in other ways.

2.2 Intended outcome

In this lab the goal is to understand the important concept of a testbench and design a testbench that verifies that a BCD converter converts an 8 bit input vector into three BCD digits correctly.

You should also understand the concept of code instrumentation and measure the start bit of the serial transmission line (RS232) when the computer is set to the serial speed of 115200 bps.

2.3 Checkpoints

In order to Pass the lab assignment the student shall be able to show that the following tasks have been fulfilled.

- Properly written VHDL implementation. Use proper indentation, spaces are cheap! Add comments when you feel it's suitable.
- A testbench where the BCD converter is instantiated as DUT, with tests covering all 255 input values and checking the output result.
- Top level of your quartus project shall instantiate the Serial UART, BCD converter and a debug process to switch the source for LED output.
- A key push button shall be used as reset in the design.
- The Serial UART component shall be edited, with a test_port output for debug.
- A debug process in Serial UART shall be written to count the number clock cyles the start bit is active.
- A switch (SW) input shall be used to toggle between debug output and BCD output.
- Design shall also be shown to work in hardware both showing the received byte value as BCD decoded data, and the number of clock cycles the start bit is detected low.

Address:
Mor Kerstins väg 27
511 53 Kinna
Sweden

Tel:
+46-703-748429

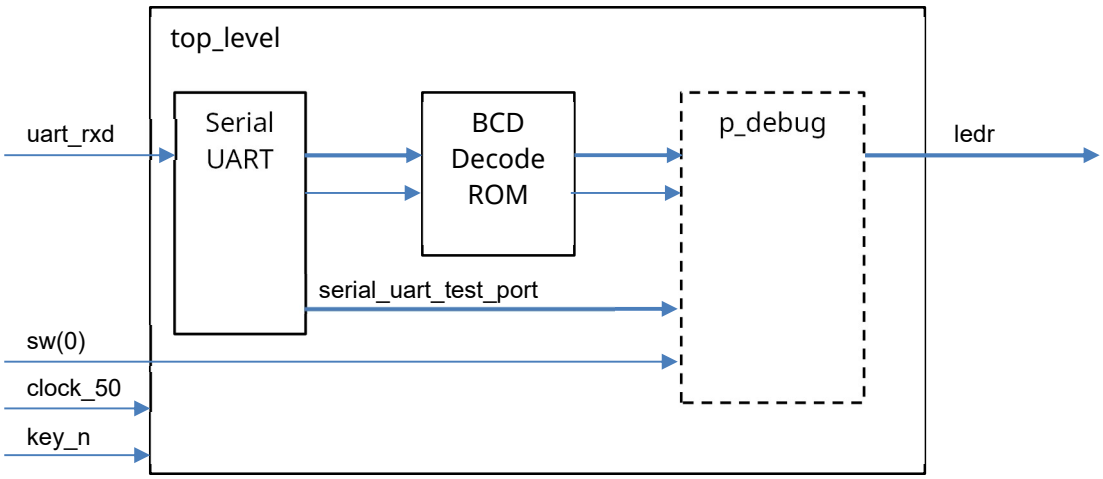
Registration no:
559037-6603

E-mail:
kent@elekteknik.se

3 Lab instructions

3.1 General

The top level design file shall be designed as shown in the block diagram below.



Serial UART and BCD Decode ROM components can be fetched from the course homepage. Study the code to ensure you understand the functionality of the code, ask if something seems unclear.

The p_debug process shall double synchronize the switch input (SW0 is suggested), when the switch input is zero the output from the BCD Decode ROM component is MUXed out on the ledr outputs, since BCD Deocde ROM component outputs 3*4 = 12 bits and some development boards only have 10 red LEDs, feel free to skip the 2 most significant bits of the bcd_2 output vector.
If the switch input is '1' the serial_uart_test_port shall be presented on the red LEDs instead.

Suggested workflow:

1. Download the testbench_top.vhd file from the course homepage and store it in the project folder. **Do not** add this to the quartus project since this file only needs to be compiled in Modelsim.

   Download the BCD Decode ROM component to be used as DUT in your testbench and as a sub component in your design top level, store it in your project folder.

2. Create a top level design (feel free to copy and edit the lab 5 project)

   Instantiate Serial UART and the BCD Decode ROM design files which can be fetched from the course homepage.

   Make sure to configure Serial UART to use 115200 bps as serial speed!

   Use the key_n input as reset, add inverter if needed and make sure polarity of reset is handled correctly (i.e. active high/low).

Address: Mor Kerstins väg 27, 511 53 Kinna, Sweden
Tel: +46-703-748429
Registration no: 559037-6603
E-mail: kent@elekteknik.se

The Serial UART received data byte shall be connected to the BCD Decode ROM input, valid out signal from Serial UART shall be connected to Valid in on the BCD Decode ROM component.

3. Write the p_debug process as specified above. This is a simple double synchronization of the switch signal used to select the source data for the LED outputs.

4. Study the Serial UART code and try to identify a way to how to detect the start bit of the serial transmission. Note that it will be impossible to detect how long the start bit is if the LSB bit of transmitted data is not '1'.
Tip is to check the state signal, bit counter and logical level of serial data.
Write a small debug process in parallel with the other processes in Serial UART component which checks the signals needed and counts clock cycles when start bit is detected low. Limit counter value to 1023, and output this counter value on the serial_uart_test_port when finished counting.

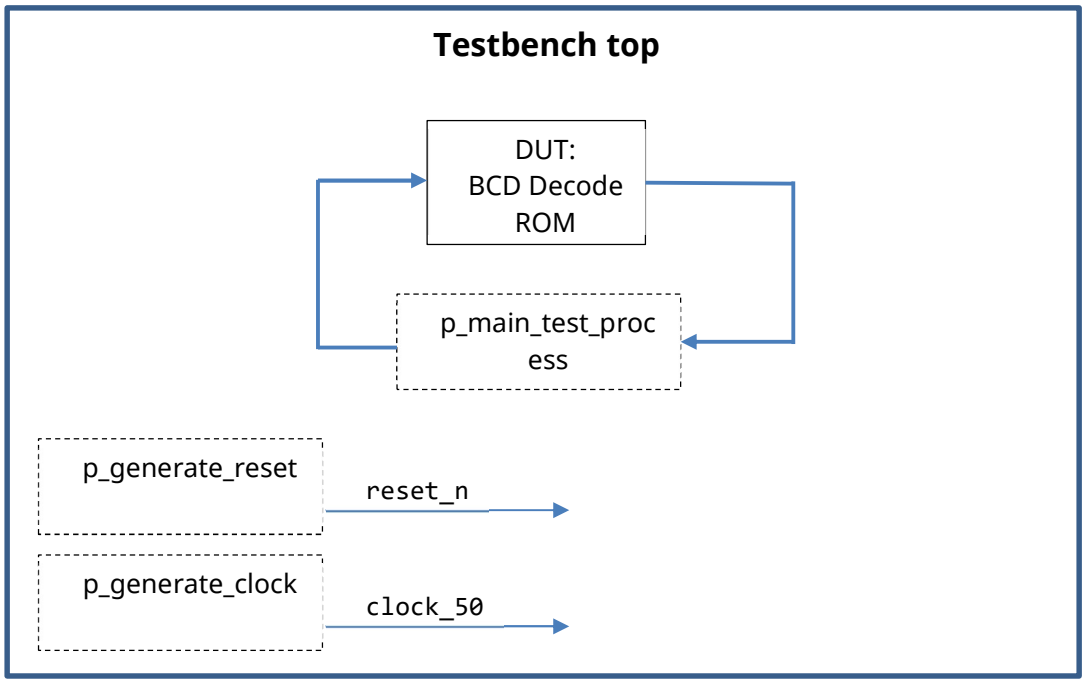
   Make sure your code is synthesizable!

5. Start up RTL simulation. You can simulate a serial transmission the same way as in lab 5 using do file for the top level if you wish (probably a good idea). Make sure that the bit time is adjusted to 115200 bps.

Running testbench for BCD Decode ROM component

When top level design is simulated end the simulation and compile the testbench file in Modelsim, see section 3.2 below for tips.

The tesbench is designed according to the sketch below.



1. Run the Simulation on the testbench as the simulation top level.
2. Edit the testbench to test all 256 possible values on the input and ensure that the test pass. If you edit the testbench_top file you can recompile it and run the restart command to restart the simulation without closing Modelsim.
3. When the testbench tests all 256 input combinations, edit the BCD Decode ROM component, where the ROM is initialized you can inject an error by setting the Boolean to the ROM initialization function to true.
4. Recompile the BCD Decode ROM component and testbench, restart the testbench and ensure that the testbench detects the introduced error.

Optional:

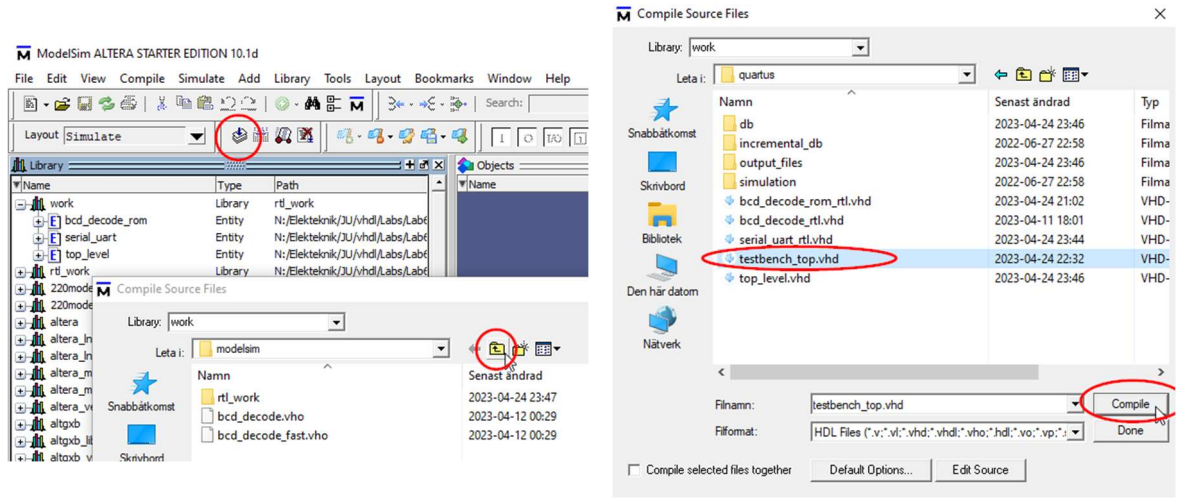
- In the synthesizable top level design, use the BCD Decode ROM outputs and convert the values to seven segment display numbers instead to present the received byte value on the 7 segment displays. The received ASCII Character from the PC should then be shown as a decimal value on the 7 segment displays at the same time as start bit time is analyzed.

Address: Mor Kerstins väg 27, 511 53 Kinna, Sweden
Tel: +46-703-748429
Registration no: 559037-6603
E-mail: kent@elekteknik.se

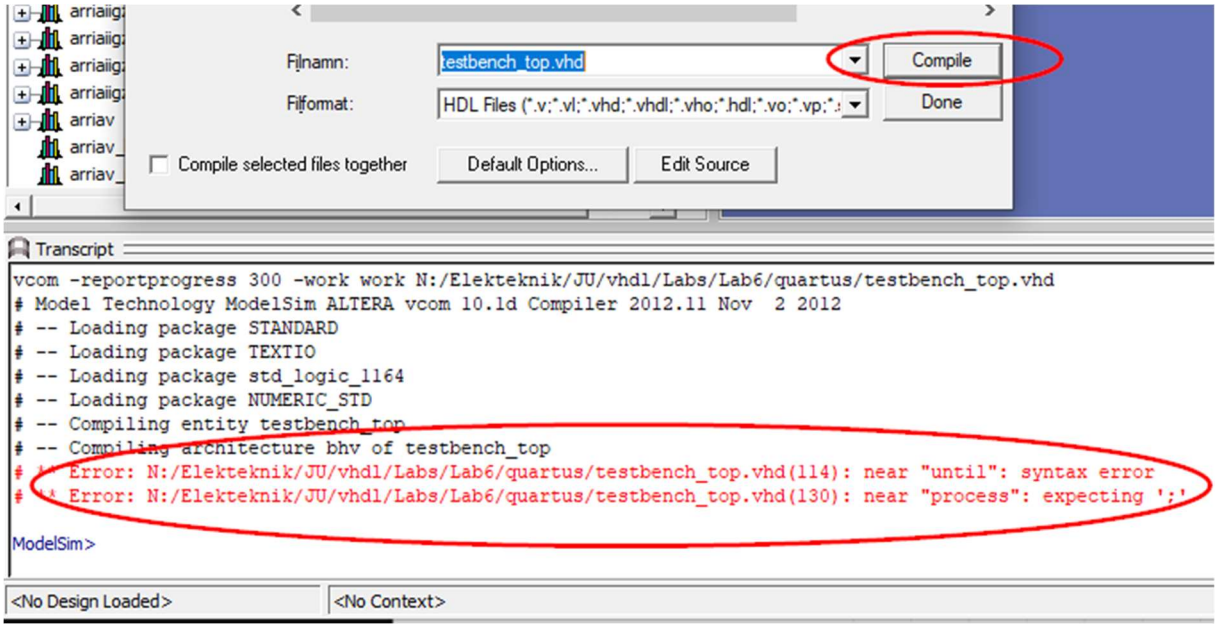
3.2 Running a testbench in Modelsim

To run a testbench in Modelsim use the compile button and compile the testbench.

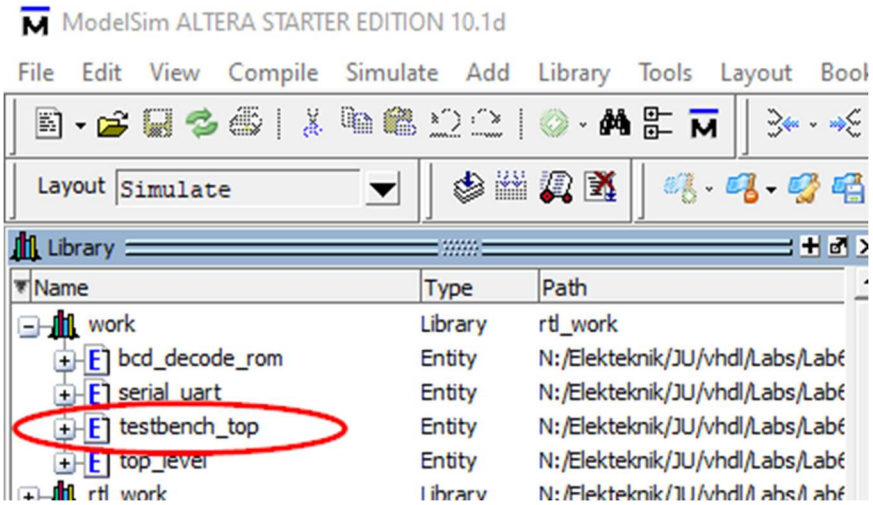
The testbench file is hopefully stored in the project folder so you will need to browse to it from the modelsim working directory, by clicking “up a level” in the folder structure.



Check Modelsim Transcript window to ensure that no compile errors is found.

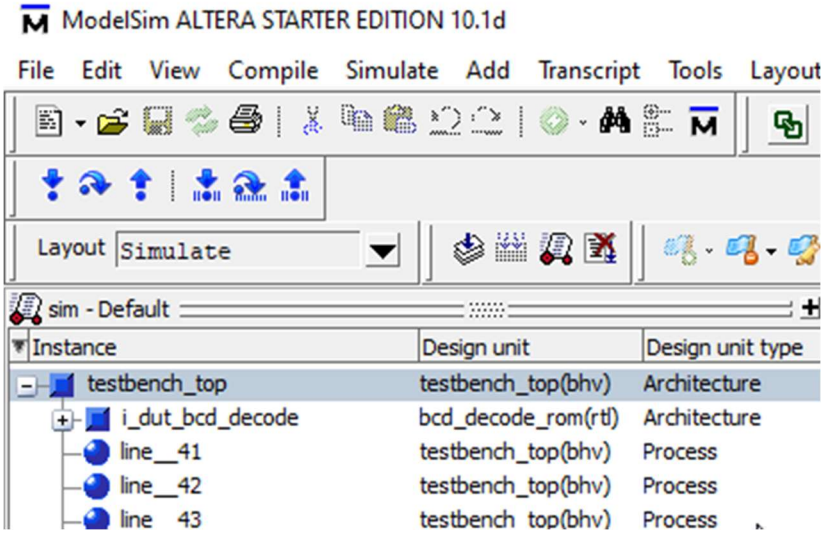


Once the testbench is compiled successfully you should be able to find it in the library “work”.



Double click the testbench top entity to start simulation.

Simulation should start and the DUT should be shown as an instance.



Drag the signals of interest (preferably all of them), as you're used to into the wave window, group them to not mix them up.

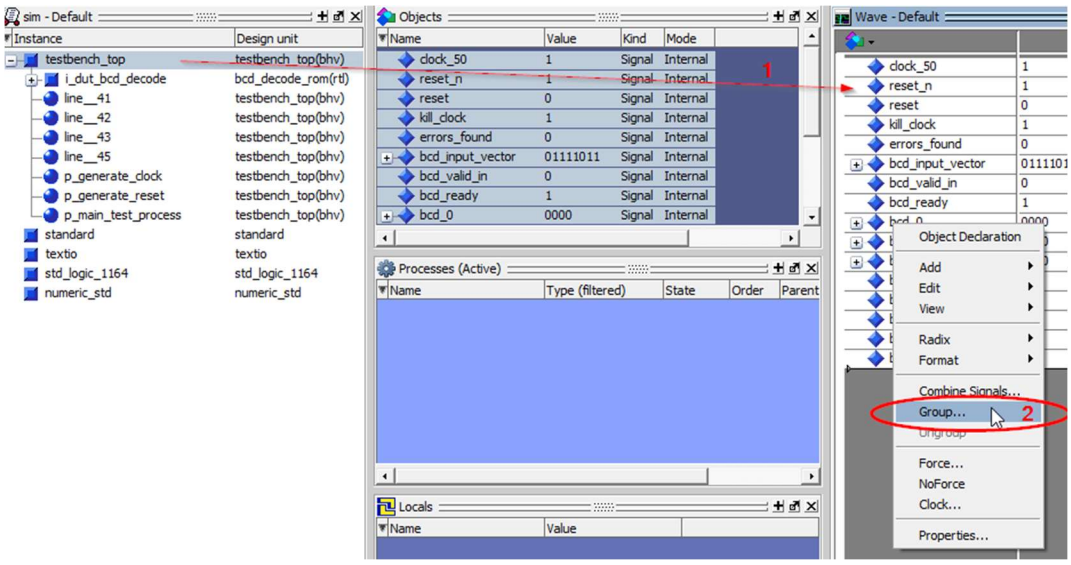
Address:
Mor Kerstins väg 27
511 53 Kinna
Sweden

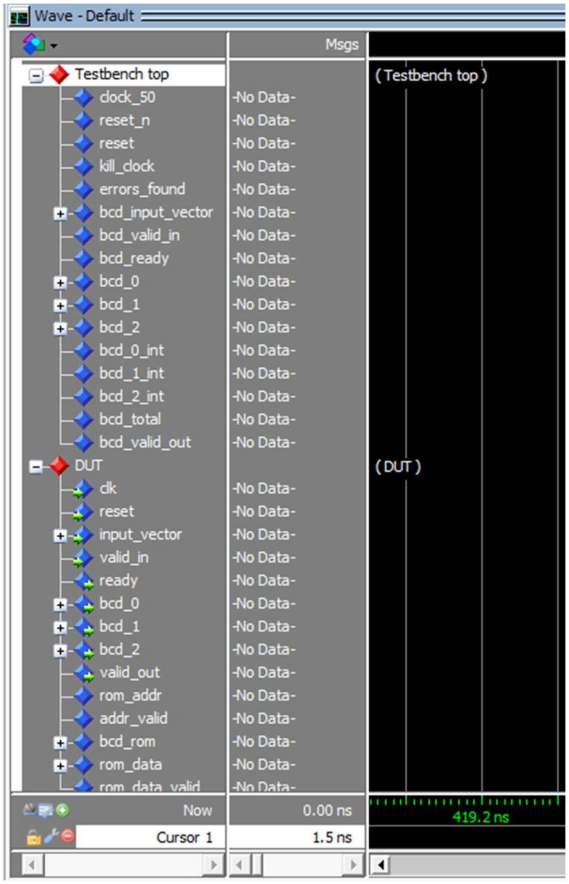
Tel:
+46-703-748429

Registration no:
559037-6603

E-mail:
kent@elekteknik.se

You group signals by right clicking marked signals in the wave window and selecting “Group…”





Name the group something smart, e.g. instance name, and repeat for all instances.

Run the full simulation with the command “run -all”

This command will run the testbench and make modelsim stop simulation when all clocks and sequential processes are stopped (see kill_clock signal in testbench file).

Address:
Mor Kerstins väg 27
511 53 Kinna
Sweden

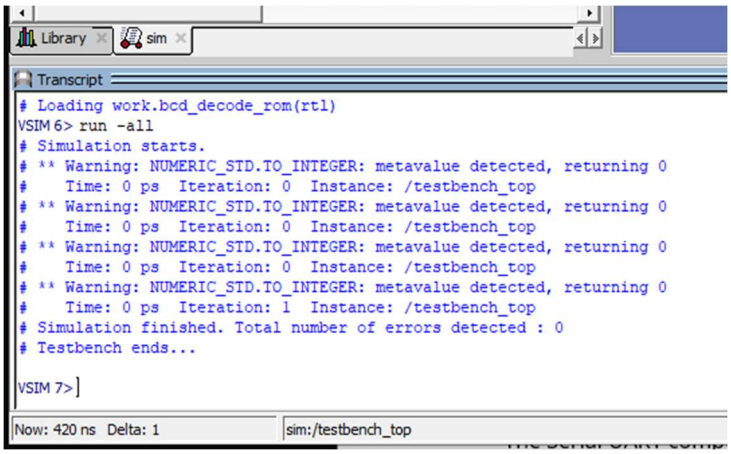
Tel:
+46-703-748429

Registration no:
559037-6603

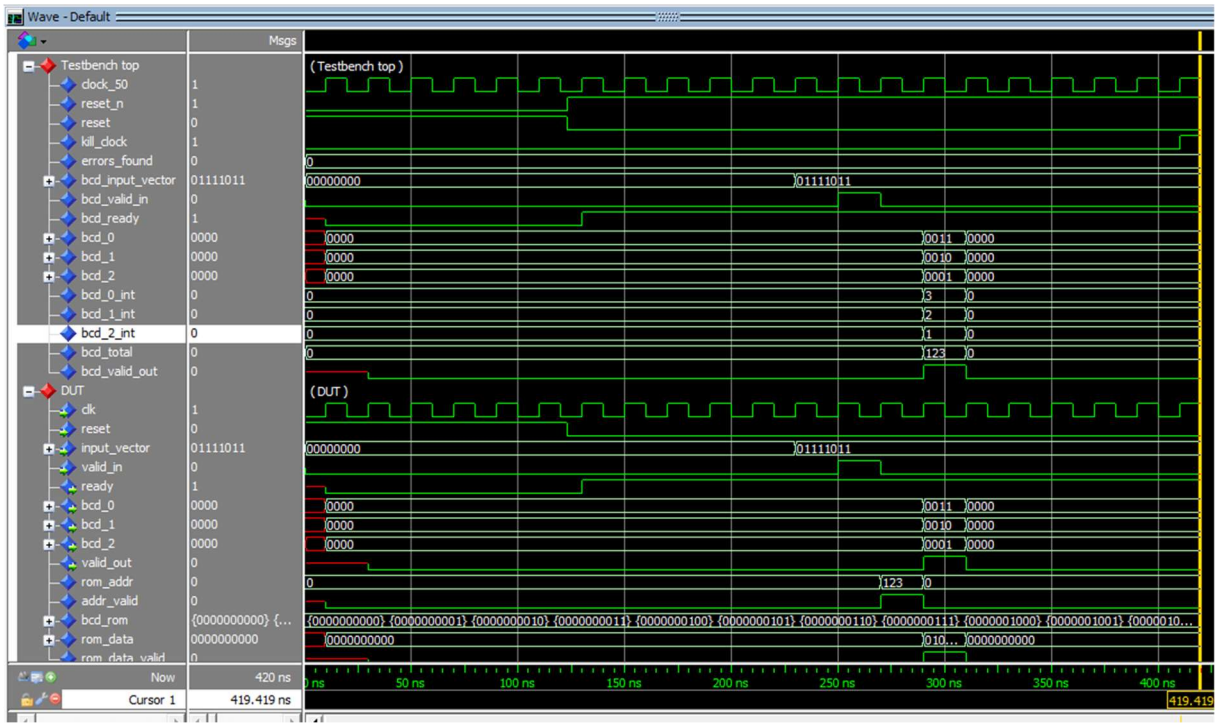
E-mail:
kent@elekteknik.se

You will probably get a few warnings early in simulation, **this is normal and can safely be ignored.**

The transcript window should hopefully look something similar to the screenshot below:



Wave window should be updated and look similar to below:



Address:
Mor Kerstins väg 27
511 53 Kinna
Sweden

Tel:
+46-703-748429

Registration no:
559037-6603

E-mail:
kent@elekteknik.se